

Spring IoC 原理和实现—复习要点

1 程序架构的发展

1.1 从 HelloWorld 到 IoC

- 阶段 1 —硬编码（经典 HelloWorld 程序）
 - 输出内容直接写在代码中，修改需要重新编译和测试 [p.7-8]
- 阶段 2 —命令行参数（HelloWorldCmdLine）
 - 程序从外部接受字符串参数，运行时决定输出内容，初步解耦 [p.8]
- 阶段 3 —面向对象与分层架构（OOP）
 - 三个功能模块：持久化数据读取 (FileHelloStr)、业务逻辑处理 (HelloWorld)、界面输出 (HelloWorldClient) [p.9-11]
 - 问题：高耦合——HelloWorld 依赖 FileHelloStr，HelloWorldClient 依赖 HelloWorld，可维护性和扩展性差，不适合大型企业级应用 [p.13]
- 阶段 4 —简单工厂模式（初步解耦）
 - 将依赖对象的创建集中到工厂中，OrderService 不再直接依赖具体实现类，只依赖接口和工厂 [p.14-15]
 - 局限：新增支付方式时仍需修改工厂代码；工厂成为新的依赖点；静态方法难以替换为 Mock 对象进行测试 [p.15]
- 阶段 5 —IoC 思想的萌芽（更高级的解耦）
 - 将“创建和管理对象”的职责完全从业务代码中抽离，交给通用基础设施（容器），通过依赖注入 (DI) 实现控制反转 [p.16]

2 IoC 的基本概念与核心原理

2.1 什么是 IoC (Inversion of Control, 控制反转)

- 面向对象编程中的一种设计原则，用于降低计算机代码之间的耦合度 [p.18]
- 最常用的实现方式是依赖注入 (Dependency Injection, DI) [p.18]
- 对象在被创建时，由系统外部实体将其所依赖的对象的引用传递（注入）进来，而不是系统内部自行创建 [p.18]
- 目前最常见的具体实现是 Spring Framework [p.18]

2.2 IoC 的核心思想与实现基础

- **核心思想**：解耦合——将组件的构建（生产）和组件的使用分开；每个组件只考虑内部的事情，明确和定义组件间的接口，实现高内聚、低耦合 [p.19]
- **实现基础**：面向接口编程 + 工厂模式 + 依赖注入 [p.19]
- **四要素**：接口的定义 → 组件的生产 → 组件的使用 → 运行时注入 [p.19]

2.3 IoC 原理范例——打印机案例

- **第一步**：定义组件接口——墨盒接口 (Ink) 和纸张接口 (Paper) [p.20-21]
- **第二步**：使用接口开发打印机（不依赖具体实现） [p.22]
- **第三步**：组装打印机——添加实现类 [p.23]
- **第四步**：使用 Spring IoC 容器装配对象 [p.24]

3 Spring IoC 容器

3.1 容器概述

- **Bean**：由 Spring IoC 容器负责实例化、组装和管理的对象称为 Bean。Bean 就是普通的 Java 对象 (POJO)，但被纳入了容器的管理范围 [p.25]

3.2 核心接口

- **BeanFactory**（顶层接口）
 - 提供最基础的 IoC 功能 [p.25]
 - 采用延迟加载策略（Lazy Loading），仅在调用 `getBean()` 时才实例化 Bean [p.25]
 - 适用于资源受限的环境（如移动设备） [p.25]
- **ApplicationContext**（BeanFactory 的子接口）
 - 企业级应用的首选 [p.25]
 - 容器启动时预加载所有单例 Bean（立即加载 / Eager Loading） [p.25]
 - 提供额外的企业级功能：国际化 (MessageSource)、事件发布 (ApplicationEvent)、AOP 集成、环境抽象 (Environment) 等 [p.25]

3.3 常见的 ApplicationContext 实现

- **ClassPathXmlApplicationContext**：从类路径加载 XML 配置文件 [p.25]
- **FileSystemXmlApplicationContext**：从文件系统加载 XML 配置文件 [p.25]
- **AnnotationConfigApplicationContext**：基于 Java 配置类和注解 [p.25]

3.4 容器使用范例

- **第 1 步**：导入 Spring IoC 相关的依赖包 [p.27]

- 第 2 步：创建 applicationContext.xml 配置文件 [p.28]
- 第 3 步：添加 Bean 的 XML 配置 [p.29]
- 第 4 步：初始化容器，使用容器装配对象 [p.30]

4 Bean 的定义和配置方式

4.1 XML 配置 (Spring 1.x)

- 优点：完全解耦，所有 Bean 关系在 XML 中定义，修改无需重新编译代码 [p.32]
- 缺点：配置冗长，维护成本高，难以管理大量 Bean [p.32]

4.2 注解配置 (Spring 2.5+)

- 类级别：@Component、@Service、@Repository、@Controller [p.32]
- 字段/方法级别：@Autowired、@Resource [p.32]
- 优点：开发效率高，配置与代码结合紧密、直观 [p.32]
- 缺点：Bean 的依赖关系分散在代码中，修改可能需要重新编译 [p.32]

4.3 Java Config (Spring 3.0+)

- 使用 @Configuration 标记配置类，@Bean 标记方法返回 Bean 实例 [p.32]
- 优点：类型安全，支持重构，可灵活编程（如条件判断创建 Bean）[p.32-33]
- Spring Boot 基于 Java Config 进一步简化了配置 [p.33]

5 依赖注入 (Dependency Injection, DI)

Spring IoC 支持三种依赖注入方式 [p.34]:

5.1 构造器注入 (Constructor Injection)

- 优点 [p.34-35]:
 - 强制性：对象必须提供依赖才能被创建，避免空指针异常
 - 不可变性：依赖一旦注入，在对象生命周期内不会改变，适合线程安全场景
 - 易于测试：在单元测试中可直接 new 对象并手动传入 mock 依赖，无需启动 Spring 容器
 - 明确依赖关系
- 缺点 [p.35]：当依赖过多时，构造器参数列表会变得很长（这是代码需要重构的“坏味道”，提示该类可能承担了过多职责）

5.2 Setter 注入

- 优点 [p.34, 36]:
 - 灵活性: 依赖可以在对象创建后重新设置 (如通过配置变更动态切换实现), 适合可选依赖
 - 避免循环依赖: 配合三级缓存可以解决部分循环依赖场景 (构造器注入对此可能直接抛出异常)
- 缺点 [p.36]:
 - 不确定性: 对象创建后依赖可能为 null (忘记调用 setter), 无法保证对象完全初始化
 - 不适合强制依赖: 如果某个依赖是必须的, 无法强制要求

5.3 字段注入 (Field Injection)

- 优点 [p.34, 37]: 代码极其简洁, 开发效率高
- 缺点 [p.37]:
 - 无法声明 final: 依赖可变, 破坏不可变性
 - 隐藏依赖: 通过类的外部接口 (构造器、setter) 看不到依赖, 容易导致运行时意外 (依赖未注入)
 - 与容器强耦合: 脱离 Spring 容器 (如单元测试) 时无法简单地通过 new 注入依赖, 必须使用反射或启动容器
 - 容易产生循环依赖: 字段注入让依赖关系不显式, 可能不经意间引入循环依赖
- 官方立场: 程序员实际使用中很常见, 但官方文档和组织标准中均不推荐 [p.34]

5.4 处理多个同类型 Bean

当一个接口有多个实现类时, 三种指定方式 [p.38-40]:

- 方法一: 使用 @Primary 标记首选实现 [p.38]
- 方法二: 使用 @Qualifier 精确指定 Bean 名称 [p.39]
- 方法三: 注入全部实现 (批量处理, 如 List<Interface>) [p.40]

6 Bean 的作用域 (Scope)

- singleton (默认)
 - 每个容器仅创建一个共享实例 [p.41]
 - 适用于无状态服务或 DAO [p.41]
 - Spring 的单例是 “容器级单例”, 每个容器一个实例, 而非 JVM 层面的单例 [p.41]
- prototype
 - 每次请求 Bean 时都会创建新的实例 [p.41]
 - 适用于有状态或可变对象, 例如表示会话信息的对象 [p.41]
 - 容器仅负责初始化, 不负责销毁; 客户代码需自行清理资源 [p.41]

- **request** (Web 环境)
 - 每个 HTTP 请求创建一个新实例，请求结束时销毁 [p.41]
- **session** (Web 环境)
 - 每个 HTTP 会话维持一个实例，会话失效或超时后销毁 [p.41]
- **application** (Web 环境)
 - 与 ServletContext 同生命周期，整个 Web 应用共享一个实例 [p.41]

7 本章小结

7.1 三个核心问题

1. 为什么需要 IoC?

- 了解程序架构随着业务和代码规模的变化而变化,掌握其中产生的各种程序架构 [p.42]
- 从硬编码 → 命令行参数 → 面向对象分层 → 简单工厂 → IoC 容器, 每一步都是对耦合问题的深入解决 [p.7-16]

2. IoC 和 Spring IoC 的核心原理

- 掌握控制反转 IoC 的核心原理 (解耦合, 将组件构建与使用分开) [p.19, 42]
- 掌握 Spring IoC 的 Bean 工厂核心机制 (BeanFactory / ApplicationContext) [p.25, 42]

3. Spring IoC 的相关概念及使用

- Bean 的定义和配置方式 (XML / 注解 / Java Config) [p.32-33, 42]
- 依赖注入 DI 的三种方式 (构造器 / Setter / 字段注入) [p.34-37, 42]
- Bean 的作用域 (singleton / prototype / request / session / application) [p.41-42]